

Overall Guide

Snapdragon Multiverse Hackathon: Integrated Developer Guide

Welcome to the Snapdragon Multiverse Hackathon. This guide provides a unified technical workflow for deploying Generative AI and Classical ML models using the Qualcomm ecosystem, specifically focusing on the LiteRT and LiteRT-LM frameworks on the Samsung Galaxy S25 Ultra.

Event Overview & Target Hardware

The hackathon aims to develop AI experiences across multiple form factors including AI PCs (Snapdragon X Series), mobile devices, and microcontrollers like the Arduino UNO Q.

Primary Device Specifications

- Device: OnePlus 15
- Chipset: Snapdragon® 8 Elite (SM8850).
- Acceleration: Hexagon NPU (v81) for high-efficiency hardware acceleration.
- Compute : Snapdragon X-elite
- IoT: Arduino UnoQ

Core Architectural Frameworks

The hackathon is divided into two primary tracks based on the model type and required runtime.

Runtime	Primary Use Case	Model Format
LiteRT-LM	Large Language Models (LLMs) and Vision-Language Models (VLMs).	<code>.litemlm</code> .
LiteRT	Classical ML (Classification, Segmentation, Object Detection).	<code>.tflite</code> .
ExecuTorch	Cross-platform Edge AI	Executable
llama.cpp with Qualcomm Backend	LLM Inference	GGUF
Arduino UNOQ	Microcontroller ML	Native
EdgeImpulse	ML Pipeline	Various
AI Hub	Optimization & Deployment	Various

Backend Stack Hierarchy

For maximum performance on Snapdragon platforms, utilize the NPU (QNN) backend.

- NPU (QNN): High-efficiency acceleration for Snapdragon 8 Elite (SM8750).
- GPU: Parallel task acceleration for standard Android performance boosts.
- CPU: Universal fallback for all devices.

Submission & Evaluation

Project Requirements

To be eligible for prizes, submissions must include:

- GitHub Repository: Containing open-source code and a detailed README.
- README Components: Description, names/emails of team members, setup instructions, and an open-source license.
- Edge Execution: The application and most components must run locally on the device.

Judging Rubric (Scored 1-10)

1. Technical Implementation (40%): Performance, latency, and resource utilization on the NPU.
2. Innovation (25%): Uniqueness and real-world problem solving.
3. Deployment (20%): Ease of installation and stability during demos.
4. Presentation (15%): Clarity and code quality.

Key Developer Resources

- Qualcomm AI Hub: aihub.qualcomm.com.
- LiteRT Samples: github.com/google-ai-edge/litert-samples.
- Hugging Face (LiteRT Community): Pre-quantized models for Gemma and other architectures.
- Support: Real-time assistance available on the Qualcomm Discord.

LiteRT

Qualcomm Hackathon: LiteRT Developer Guide

This technical guide provides a comprehensive workflow for developers deploying the Generative AI and Classical ML models using LiteRT and LiteRT-LM.

Table of Contents

[Google x Qualcomm Hackathon: Developer Guide](#)

[Pre-work](#)

[Core Architectural Concepts](#)

[LiteRT vs. LiteRT-LM](#)

[Backend Stack Hierarchy](#)

[Hackathon Tracks](#)

[Models](#)

[Additional References](#)

- [Device Details](#)
- [Track 1: LiteRT-LM for Generative AI \(e.g. Gemma3 1B & FastVLM\)](#)
- [Track 2: LiteRT for Classical Models](#)
- [Details for Leverage Google AI Edge Gallery App](#)

Pre-work

Suggest participants try out the samples below to get acquainted with LiteRT and LiteRT-LM

1. Try out the Qualcomm LiteRT & LiteRT-LM [sample apps](#) with LiteRT and LiteRT-LM. Additional samples to learn more about LiteRT CPU/GPU/NPU(jit) can be found here: [Sample Apps](#)
 - Models used in the sample apps can be found on the following Hugging Face collection:
<https://huggingface.co/collections/litert-community/qualcomm>
2. Go over the documentation
 - <https://ai.google.dev/edge/litert-lm> for Track 1
 - <https://ai.google.dev/edge/litert> for Track 2

Core Architectural Concepts

Understanding the relationship between the runtime and its specialized wrapper is essential for efficient deployment.

LiteRT vs. LiteRT-LM

- LiteRT: Serves as the foundational high-performance runtime for on-device machine learning, executing .tflite models across various hardware.
- LiteRT-LM: A specialized high-level API wrapper powered by LiteRT specifically for Large Language Models (LLMs) and Vision-Language Models (VLMs). It automates complex tasks such as tokenization, KV-caching, and multimodal input handling. LiteRT-LM executes .litemlm files.

Backend Stack Hierarchy

LiteRT optimizes performance by offloading computation to available hardware:

Backend	Capability	Optimal Use Case
CPU	Universal fallback for all devices	Low-complexity models or non-accelerated hardware.
GPU	Parallel task acceleration	Standard Android performance boost.
NPU (QNN)	High-efficiency hardware acceleration	Snapdragon 8 Elite (SM8750) for maximum speed.

Track 1: LLM-Based Consumer Use Journeys with LiteRT-LM

Build on-device LLM experiences using Generative AI models with a focus on real consumer use cases.

What’s provided:

- Ready-to-use the Generative AI models, Gemma, EmbeddingGemma, and FastVLM, available via [LiteRT Hugging Face](#)
- A reference [sample app](#) for Gemma 4 2B model

This track is ideal for developers interested in integrating Generative AI-powered features into Android applications with on-device inference.

Track 2: Classical Models — Vision & Audio with LiteRT

Build efficient, on-device AI applications using classical ML models for vision or audio workloads.

What's provided:

- Ready-to-use models that do not require quantization or conversion
- NPU-friendly models available via [Qualcomm AI Hub](#) and [LiteRT HuggingFace](#)
- A sample application available through the LiteRT repository under the LiteRT-Samples github repo Qualcomm [directory](#) (mobilenet v2, object_detection), [EmbeddingGemma](#), and [image segmentation](#)

This track is ideal for developers looking to deploy performant vision or audio pipelines on Snapdragon NPUs with minimal overhead.

Models

<https://huggingface.co/collections/litert-community/qualcomm>

Track 1: Generative Models via LiteRT-LM

[Gemma4 E2B](#) ([Sample app for NPU](#) / [Sample app for CPU/GPU](#))

Gemma 4 E2B is a multimodal "edge" model released by Google DeepMind in April 2026. The "E" stands for Effective parameters, meaning it uses a specialized architecture to achieve high performance with a low memory footprint suitable for local, on-device deployment.

- gemma-4-E2B-it_qualcomm_sm8750.litertlm: Specifically compiled for the Snapdragon 8 Elite NPU.

- gemma-4-E2B-it.litertlm: Designed for standard GPU and CPU acceleration.

[FastVLM-0.5B](#) (Sample app for CPU/GPU/NPU)

FastVLM is a multimodal vision-language model capable of interpreting image data and generating text responses.

- FastVLM-0.5B.qualcomm.sm8750.litertlm Specifically compiled for the Snapdragon 8 Elite NPU.

[Gemma3 1B](#)

Gemma 3 1B is a lightweight, open-weight language model from Google, optimized for on-device, mobile, and web applications. Released in March 2025, this 1-billion-parameter model offers high efficiency for text-based tasks.

- Gemma3-1B-IT_q4_ekv1280_sm8750.litertlm Specifically compiled for the Snapdragon 8 Elite NPU.
- Gemma3-1b-it-int4.litertlm Designed for standard GPU and CPU acceleration.

[Function Gemma](#)

FunctionGemma is a specialized 270M parameter model based on Gemma 3, fine-tuned for efficient function calling and API action generation in local, on-device AI applications. It converts natural language into structured function calls, providing high performance for mobile/IoT devices with low latency. (Gallery App [mobileactions](#), [tinygarden](#))

- Mobile_actions_q8_ekv1024.litertlm Designed for standard GPU and CPU acceleration.

Track2: LiteRT, Vision and Audio (aka: traditional/classical models)

For more models: <https://aihub.qualcomm.com/apps?os=Android>

Models / Demo Page	CPU/GPU/NPU (JIT)	Sample App
EmbeddingGemma-300m	Supported	Sample App
Image Segmentation	Supported	Sample App (CPU/GPU) Sample App (NPU)
Image Classification	Supported	Sample App
Object Detection	Supported	Sample App

Note: Migrating guide from TensorFlow Lite to LiteRT - [link](#)

Additional References

Applicable if you are not starting with a sample app.

Device Details

- Type: Mobile
- Name: One Plus 15
- Chipset: Snapdragon® 8 Elite Gen 5
- Hexagon Version and SOC Model: v81 and 69, respectively

Track 1: LiteRT-LM for Generative AI (e.g, Gemma4 E2B & FastVLM)

This guide details the LiteRT-LM ([repo](#)) Developer Workflow for implementing machine learning inference in Android applications using the CompiledModel API. The following documentation provides instructions and code snippets for setting up and running models using various hardware accelerators, including CPU, GPU, and NPU, and outlines the core steps for creating an environment, configuring options, and performing inference.

The development workflow can be separated into the following two major entry points: 1) Use our Kotlin API with Maven package; 2) Use our C++ API and build from

source

Kotlin API with Maven Package

Please follow the [instructions](#) in the official documentation to set up your project for importing the LiteRT-LM Maven package.

To run the models on NPU, please make sure you copy the NPU libraries from [the sample app](#) to your app folder. The required libraries are:

- `libLiteRtDispatch_Qualcomm.so`
- `libQnnHtp.so`
- `libQnnHtpV81Skel.so`
- `libQnnHtpV81Stub.so`
- `libQnnSystem.so`

Separately, you'd also need to copy the prebuilt `libGemmaModelConstraintProvider.so` from the [LiteRT-LM/prebuilt](#) folder (from the corresponding platform) to your app folder.

This [repo](#) gives you an example sample app that uses the Gemma4 model with NPU.

1. Setting Up the Engine

The *Engine* is the brain that runs your model. You need to provide a configuration that tells it where your model file is located and which hardware (like the NPU for speed) to use for both text and images.

```
Kotlin
val config = EngineConfig(
    modelPath = "/path/to/gemma-4-E2B-it_qualcomm_sm8750.litertlm",
    backend = Backend.NPU(nativeLibDir), // Use NPU for text
    visionBackend = Backend.GPU(),       // Use GPU for images
    audioBackend = Backend.CPU()         // Use CPU for audio
)
val engine = Engine(config)
engine.initialize()
```

2. Creating a Conversation

A *Conversation* object helps the AI remember what was said earlier in the chat. You create it directly from your initialized engine.

Kotlin

```
val conversation = engine.createConversation()
```

3. Sending Messages with Images

Finally, you can send a message that includes both an image and a text question. The AI will process these together and send back the response in small pieces (chunks) that you can print or show in your app.

Kotlin

```
val userMessage = Message.user(
    Contents.of(
        Content.ImageFile("/path/to/photo.jpg"),
        Content.Text("What is in this image?")
    )
)
conversation.sendMessageAsync(userMessage).collect { response ->
    val chunk = response.contents.contents
        .filterIsInstance<Content.Text>()
        .joinToString("") { it.text }
    println(chunk)
}
```

C++ API and Build from Source

Please follow the [instructions](#) in the official documentation to set up the environment and build the program. Next, use the following code snippets to start playing with it.

C/C++

```
#include "runtime/engine/engine.h"

// ...
```

```

// 1. Define model assets and engine settings.
auto model_assets = ModelAssets::Create(model_path);
CHECK_OK(model_assets);

auto engine_settings = EngineSettings::CreateDefault(
    model_assets,
    /*backend=*/litert::lm::Backend::CPU);

// 2. Create the main Engine object.
absl::StatusOr<std::unique_ptr<Engine>> engine =
    Engine::CreateEngine(engine_settings);
CHECK_OK(engine);

// 3. Create a Conversation
auto conversation_config =
    ConversationConfig::CreateDefault(**engine);
CHECK_OK(conversation_config)
absl::StatusOr<std::unique_ptr<Conversation>> conversation =
    Conversation::Create(**engine, *conversation_config);
CHECK_OK(conversation);

// 4. Send message to the LLM with blocking call.
absl::StatusOr<Message> model_message = (*conversation)->SendMessage(
    JsonMessage{
        {"role", "user"},
        {"content", "What is the tallest building in the world?"}
    });
CHECK_OK(model_message);

// 5. Print the model message.
std::cout << *model_message << std::endl;

// 6. Send message to the LLM with asynchronous call
// where CreatePrintMessageCallback is a users implemented callback
// that would
// process the message once a chunk of message output is received.
std::stringstream captured_output;

```

```

(*conversation)->SendMessageAsync(
    JsonMessage{
        {"role", "user"},
        {"content", "What is the tallest building in the world?"}
    },
    CreatePrintMessageCallback(std::stringstream& captured_output)
);
// Wait until asynchronous finish or timeout.
*engine->WaitUntilDone(absl::Seconds(10));

```

Track 2: LiteRT for Classical Models

(e.g., Image Segmentation)

The LiteRT CompiledModel API is used for smaller, classical machine learning models (like image classification or segmentation) that don't require the specialized features of LiteRT-LM (Type 1). This is the standard way to deploy **.tflite** models.

Using CompiledModel API with CPU/GPU acceleration

This section shows how to use LiteRT's CompiledModel API with CPU and GPU acceleration. It talks about the two most common Android integration paths:

1. Kotlin / Android runtime (com.google.ai.edge.litert:litert)
2. Android NDK / C++ with the prebuilt LiteRT C++ SDK

CompiledModel is a runtime entry for LiteRT. In the current API, the core flow is:

1. Create one shared Environment.
2. Create Options and select CPU or GPU acceleration.
3. Configure backend-specific CpuOptions or GpuOptions if needed.
4. Create a CompiledModel.
5. Create input/output TensorBuffers and run inference.

1. Setup and Dependencies

First, you need an Android project (which requires Android Studio and Kotlin knowledge).

Gradle Dependency

Add the core LiteRT library to your `build.gradle.kts` file. This handles both CPU and GPU acceleration by default for Kotlin users.

```
Kotlin
dependencies {
    // Core LiteRT library
    implementation("com.google.ai.edge.litert:litert:2.1.4")
}
```

2. Android release artifacts to use for v2.1.4

For Android apps using the LiteRT Android binding, use:

```
implementation("com.google.ai.edge.litert:litert:2.1.4")
```

This is pretty much the simplest way to include GPU support on Android. For Kotlin users, the GPU accelerator is already bundled in the LiteRT Maven package.

You can then select CPU vs GPU in Kotlin by using `CompiledModel.Options(...)`, see the following code snippet

```
Kotlin
val env = Environment.create()

val cpuModel =
    CompiledModel.create(
        context.assets,
        "model.tflite",
        CompiledModel.Options(Accelerator.CPU),
        env,
    )

val gpuModel =
    CompiledModel.create(
```

```
context.assets,  
    "model.tflite",  
    CompiledModel.Options(Accelerator.GPU),  
    env,  
)
```

3. Model Loading and Configuration

Models like image segmentation typically use a standard `.tflite` format. You must load this model and select your desired hardware accelerator (CPU, GPU, or NPU).

Model Loading (Kotlin)

```
Kotlin  
  
import com.google.ai.edge.litert.CompiledModel  
import com.google.ai.edge.litert.Accelerator  
  
// 1. Select the accelerator  
val options = CompiledModel.Options.builder()  
    .setAccelerator(Accelerator.GPU) // Choose GPU for a standard  
    speed boost  
    .build()  
  
// 2. Load the model from your app's assets folder  
// The model must be packaged in your project (e.g., in the  
// 'assets' folder).  
val compiledModel = CompiledModel.create(  
    context.assets,  
    "segmentation_model.tflite", // Replace with your model file  
    name  
    options  
)
```

4. Running Inference

The execution flow involves preparing input data, creating buffers, running the model, and reading the results. Crucially, heavy operations like this should always be run off the main

application thread (e.g., using Kotlin Coroutines).

The basic execution flow is the same for CPU and GPU once the model has been Compiled.

CPU / GPU / NPU execution & Running Inference

Kotlin

```
// 1. Load model and initialize runtime.
val model =
    CompiledModel.create(
        context.assets,
        "model/mymodel.tflite",
        CompiledModel.Options(Accelerator.CPU, Accelerator.GPU)
    )

// 2. Pre-allocate input/output buffers
val inputBuffers = model.createInputBuffers()
val outputBuffers = model.createOutputBuffers()

// 3. Fill the first input
inputBuffers[0].writeFloat(...)

// 4. Invoke
model.run(inputBuffers, outputBuffers)

// 5. Read the output
val outputFloatArray = outputBuffers[0].readFloat()
```

Setup with NPU acceleration

To run classic models on NPU with JIT mode, please make sure you copy the NPU libraries from [the sample app](#) to your app folder. The required libraries are:

- libLiteRtCompilerPlugin_Qualcomm.so
- libLiteRtDispatch_Qualcomm.so
- libQnnHtp.so
- libQnnHtpPrepare.so
- libQnnHtpV81Skel.so
- libQnnHtpV81Stub.so
- libQnnSystem.so

In production, it's recommended to use Google Play Feature Delivery to ship NPU libraries

only to targeted devices. See <https://ai.google.dev/edge/litert/next/npu> for the instructions.

Create CompiledModel for Inference with NPU

```
Kotlin
val env = Environment.create(BuiltinNpuAcceleratorProvider(context))

// For JIT models, create CompiledModel with model path directly
val jitCompiledModel = CompiledModel.create(
    "/path/to/my_jit_model.tflite",
    CompiledModel.Options(Accelerator.NPU),
    env,
)

// then run inference
```

Details for Leverage Google AI Edge Gallery App

Users can clone Google AI Edge and build their app based on the following instructions.

Clone the Google AI Edge to build your own app

Detailed documentation can be found in the [LiteRT-LM Kotlin README](#), but you can also start by [cloning the Google AI Edge Gallery app code](#) that's built on top of LiteRT-LM Kotlin APIs.

Step 1: Clone the Repository

First, grab the source code from GitHub. Open your terminal and run the following command:

```
Shell
git clone git@github.com:google-ai-edge/gallery.git
```

This will create a local copy of the repository in a directory named gallery.

Step 2: Navigate to the Android Source

The Android project is located within the `Android/src/` directory of the repository. Change your current directory to this location:

```
Shell
cd gallery/Android/src/
```

Step 3: Build and Install

Now, we'll use the Gradle wrapper to build the debug version of the app and install it directly onto your connected device. Run:

```
Shell
./gradlew installDebug
```

Gradle will take care of downloading dependencies, compiling the code, and deploying the APK. Once finished, you should see "Edge Gallery" appearing in your app drawer.

Step 4: Link to Gallery App Gemma1B CPU/GPU/NPU

<https://github.com/google-ai-edge/gallery/blob/main/Android/src/app/src/main/java/com/google/ai/edge/gallery/ui/llmchat/LlmChatModelHelper.kt#L168>

Executorch

ExecuTorch: Developer Guide

Overview

This technical guide provides a comprehensive workflow for developers participating in the Qualcomm x Meta ExecuTorch Hackathon, covering model compilation, Qualcomm backend integration, Android deployment, and device-specific optimization.

Table of Contents

1. Core Architectural Concepts

- Why ExecuTorch?
- PyTorch to ExecuTorch Workflow
- Backend Stack Hierarchy
- ExecuTorch Runtime Overview

2. Qualcomm AI Runtime (QAIRT)

- Workflow
- Setup
- Building ExecuTorch

3. Models & Deployment Strategies

- Model Compilation
- Examples

4. Android Development Workflow

- Build Android Runtime
- Integrate Runtime
- Load Model
- Run Inference
- Testing

5. Additional Resources

Core Architectural Concepts

Why ExecuTorch?

ExecuTorch is PyTorch's on-device inference framework designed to bring AI models from development to deployment across mobile, edge, and embedded devices.

It enables developers to:

- Build and train models in PyTorch
- Export models into an optimized runtime format
- Execute models efficiently on-device

Benefits

- Low memory overhead
- Minimal runtime dependencies
- Responsive AI experiences
- Hardware-specific acceleration
- Local inference execution

For Snapdragon-powered devices, ExecuTorch integrates with Qualcomm's QAI RT API to leverage dedicated AI hardware, enabling:

- Improved performance
- Lower latency
- Greater power efficiency

This allows developers to build applications that benefit from:

- Real-time inference
- Offline operation
- Privacy-preserving processing

Applicable workloads include:

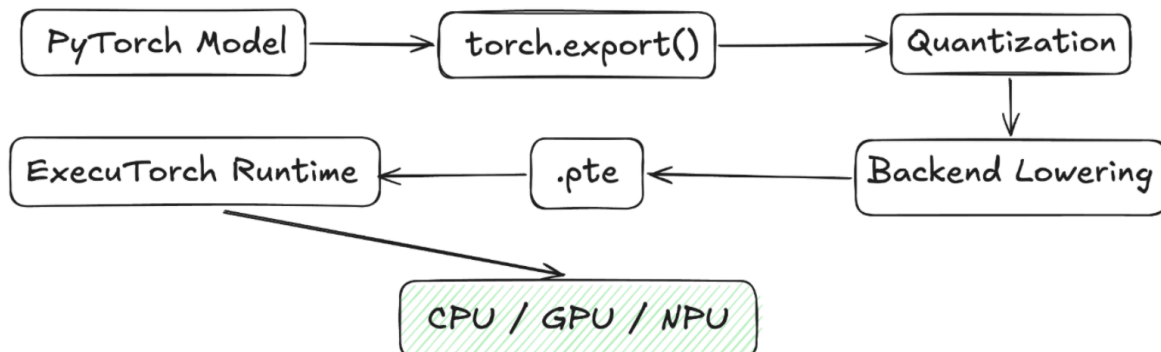
- Computer vision
- Multimodal AI
- Generative AI assistants

ExecuTorch provides a streamlined path from PyTorch model development to optimized deployment on edge devices.

PyTorch to ExecuTorch Workflow

ExecuTorch starts from native PyTorch models and produces hardware-specific `.pte` artifacts that are executed by the ExecuTorch Runtime.

Workflow



Understanding the relationship between export, lowering, and runtime execution is essential to understanding the workflow.

Backend Stack Hierarchy

ExecuTorch supports multiple acceleration backends.

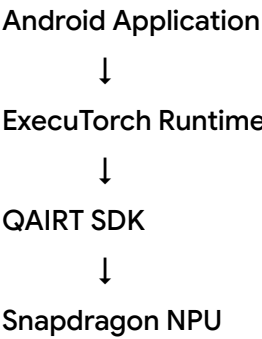
Backend	Capability	Optimal Use Case
XNNPACK CPU	Universal fallback	Maximum compatibility
Vulkan GPU	Parallel acceleration	GPU execution
Qualcomm QNN NPU	Snapdragon acceleration	Highest performance and efficiency

Recommendation

For Snapdragon devices, Qualcomm QNN is the preferred backend.

ExecuTorch Runtime Overview

Recommended architecture:



ExecuTorch x Qualcomm AI Runtime (QAIRT)

Workflow

1. Build AOT components on the host machine.
2. Push compiled models to target device (OnePlus 15).

3. Run inference on the device.

Documentation

<https://docs.pytorch.org/executorch/stable/backends-qualcomm.html>

Setup

Environment Setup

Create and activate a virtual environment.

Clone Repository

```
git clone https://github.com/pytorch/executorch.git  
cd executorch
```

Sync and Initialize Submodules

```
git submodule sync  
git submodule update --init --recursive
```

Install Dependencies

```
pip install -r requirements.txt  
pip install -r requirements-examples.txt  
pip install -e .
```

Optional Downloads

Download QAI RT SDK

Download and install the Qualcomm AI Runtime SDK.

Download Android NDK v26

Download Android NDK version 26.

Configure Environment Variables

```
export QNN_SDK_ROOT=<path_to_qnn_sdk>/2.46.0.260424/
```

```
export ANDROID_NDK_ROOT=<path_to_android_ndk>
```

Important: Android NDK 26 is required.

```
export LD_LIBRARY_PATH=$QNN_SDK_ROOT/lib/<specific tool chain>/:$LD_LIBRARY_PATH
```

Build ExecuTorch

Pathway 1 — Using Your Favorite Coding Agent

Use the following prompt:

Build ExecuTorch with Qualcomm backend support

Pathway 2 — Build Script

```
./backends/qualcomm/scripts/build.sh
```

Models & Deployment Strategies

Compiling Models

Key Arguments

Build Output Folder

`--build_path build-android`

- Created during ExecuTorch build process.

SoC Model

`--soc_model SM8850 (v81)`

- One Plus 15 = SM8850

Model Path

`--encoder_model <model_to_compile>`

Compile Only

`--compile_only`

- Generates only the `.pte` artifact.
-

Examples

GitHub Example Scripts:

https://github.com/pytorch/executorch/tree/main/examples/qualcomm/oss_scripts

Android Development Workflow

Build Android Runtime

```
./scripts/build_android_library.sh
```

Integrate Runtime (Using AAR)

Add the generated AAR to your Android project:

```
implementation(files("libs/executorch.aar"))
```

Load Model

```
Module module =  
    Module.load("MODEL.pt");
```

Run Inference

```
module.forward(...);
```

Testing

Validate application and model performance using the following metrics:

- Cold start time
 - Inference latency
 - Memory usage
 - Battery impact
-

Additional Resources

ExecuTorch Documentation

<https://docs.pytorch.org/executorch/>

PyTorch Export Documentation

<https://pytorch.org/docs/stable/export.html>

Qualcomm AI Hub

<https://aihub.qualcomm.com/>

ExecuTorch GitHub Repository

<https://github.com/pytorch/executorch>

Device Details

Device Information

Attribute	Value
Type	Mobile
Device Name	One Plus 15
Chipset	Snapdragon® 8 Elite Gen 5
SoC Model	SM8850
Hexagon Version	v81
SOC Version	69

End-to-End Summary

PyTorch Model



`torch.export()`



Quantization



Qualcomm Backend Lowering (QNN)



.pte Artifact



ExecuTorch Runtime



QAI RT SDK



Snapdragon NPU (OnePlus 15)



On-device Inference

Uno Q

Arduino UNO Q: Developer Guide

This technical guide provides a comprehensive workflow for developers building dual-brain edge applications on the Arduino UNO Q, combining a Linux microprocessor (MPU) and a real-time microcontroller (MCU) through Arduino App Lab.

Table of Contents

- Arduino UNO Q Hackathon: Developer Guide
 - Pre-work
 - Core Architectural Concepts
 - MPU vs. MCU
 - The Bridge (RPC) and Router
 - Apps and Bricks
 - Hackathon Tracks
 - Track 1: AI & Vision/Audio Apps (MPU + Bricks)
 - Track 2: Real-Time Hardware & Sensing (MCU + Bridge)
 - Device Details
 - Development Workflows
 - Workflow A: Arduino App Lab (recommended)
 - Workflow B: Arduino IDE / CLI (MCU only)
 - Workflow C: Standalone Linux (SBC mode)
 - Additional References
-

Pre-work

Before the event, participants should get acquainted with the board and Arduino App Lab:

- Download and install Arduino App Lab for your OS from the official page: <https://www.arduino.cc/en/uno-q> (Linux Debian "Bookworm"+/Ubuntu 22.04+, plus macOS and Windows builds). App Lab also comes pre-installed on the UNO Q itself.
- Read the getting-started material:
 - Product page and overview: <https://www.arduino.cc/en/uno-q>
 - Hardware documentation: <https://docs.arduino.cc/hardware/uno-q/>
 - UNO Q User Manual: <https://docs.arduino.cc/tutorials/uno-q/user-manual/>

- App Lab quickstart / tutorial:
<https://docs.arduino.cc/software/app-lab/tutorials/getting-started/>
- Run the built-in Blink LED example (see Workflow A). It is the "Hello World" of the dual-processor architecture: the Python side (MPU) toggles state and logs it, while the sketch (MCU) drives the physical LED — connected through the Bridge.
- Browse the built-in Examples and Bricks inside App Lab to see what's available out of the box.

About Arduino App Lab

Arduino App Lab is the unified development environment and workspace for the UNO Q, designed to seamlessly combine C++ Arduino sketches, Python scripts, and AI models into ready-to-run apps. It handles the orchestrated management of both the Linux-based MPU and the Zephyr-based MCU environments, ensuring tight integration across the dual-brain architecture.

The platform simplifies modular development through the use of Bricks, which allow developers to add complex features like Camera, Object Detection, and Web Servers with ease. Furthermore, the live console provides a centralized interface for real-time debugging and tracing of both MPU and MCU execution logic.

Core Architectural Concepts

Understanding the relationship between the two processors and how they talk to each other is essential for efficient development.

MPU vs. MCU

The UNO Q is a single board carrying two compute domains in the classic UNO form factor:

- MPU (Microprocessor Unit): Qualcomm Dragonwing™ QRB2210 running a full Debian Linux OS. This is where you run Python, AI models, computer vision, audio processing, networking, and web services. Think of it as a Raspberry Pi-class Linux computer on the board.
- MCU (Microcontroller Unit): STMicroelectronics STM32U585 (Arm® Cortex®-M33) running Arduino Sketches over Zephyr OS. This handles deterministic, real-time control: reading sensors, driving actuators, GPIO, and timing-sensitive tasks — the classic Arduino experience in C/C++.

The strength of the platform is combining both: heavy processing on the MPU, precise hardware control on the MCU.

The Bridge (RPC) and Router

The two processors communicate through a built-in RPC (Remote Procedure Call) system, exposed to developers as the Bridge library and managed by the arduino-router service.

- One side registers a function it wants to expose (service discovery); the other side calls it remotely as if it were local.
- Directions supported: MPU → MCU, MCU → MPU, and Linux ↔ Linux (one Linux process calling another).
- Resource lock: the Router exclusively claims the serial interface linking the two chips (e.g. `Serial1` on the Arduino/Zephyr side). Do not access these directly in your own code or the Bridge will fail.
- Linux processes reach the Router via a Unix Domain Socket; advanced users can speak the protocol directly with `msgpack` without the App Lab helper classes.

Apps and Bricks

App: a complete, self-contained project that coordinates both processors. Its standard structure is:

```
my_app/
├── README.md
├── app.yaml
├── python/
│   ├── main.py
│   └── requirements.txt
└── sketch/
    ├── sketch.ino
    └── sketch.yaml
```

- Filenames and directories matter — the UNO Q runtime looks for `python/` and `sketch/` with these exact names.
- Bricks: modular, plug-and-play software components you drop into an App to add ready-made functionality — e.g. Camera, Object Detection, Web Server, and other AI/vision/audio and integration blocks. Bricks are the fastest way to add complex features during a hackathon.

Hackathon Tracks

Participants choose one of the following tracks. Both are built on the same App Lab workflow; they differ in where the "center of gravity" of your project sits.

Track 1: AI & Vision/Audio Apps (MPU + Bricks)

Build intelligent applications that sense and react to their environment using the Linux side and AI Bricks.

What's provided / ideal for:

- Python running on the QRB2210 MPU, with Camera and Object Detection Bricks and other AI models available in App Lab.
- Optional USB camera / MIPI-CSI camera, USB microphone, and a display via USB-C dongle.
- Best for: AI-powered vision, audio classification, smart-home logic, dashboards/web interfaces that also drive physical outputs through the MCU.

Typical shape: Python (MPU) does detection/inference and web/API work → sends results over the Bridge → sketch (MCU) reacts on real hardware (LEDs, motors, relays, displays).

Track 2: Real-Time Hardware & Sensing (MCU + Bridge)

Build responsive, hardware-centric projects where the STM32 MCU does deterministic control and the MPU adds logic, connectivity, or logging.

What's provided / ideal for:

- Full Arduino shield + library compatibility (traditional UNO headers), Qwiic connector for Modulino nodes, and SPI/I²C/UART peripherals.
- Best for: sensor hubs, motor/robotics control, IoT prototypes, data logging, and anything needing tight timing that then hands data up to Linux for storage, networking, or visualization.

Typical shape: Sketch (MCU) reads sensors / controls actuators → exposes or calls RPC functions over the Bridge → Python (MPU) logs, serves a web UI, or pushes data to the cloud.

Device Details

Item	Specification
Type	Single-Board Computer (UNO form factor)
Name	Arduino UNO Q (ABX00162 / ABX00173)
MPU	Qualcomm Dragonwing™ QRB2210 — Quad-core Arm® Cortex®-A53 @ 2.0 GHz, Adreno GPU, 2× ISP (13 MP + 13 MP or 25 MP) @ 30 fps
MPU OS	Debian Linux (upstream support)
MCU	STMicroelectronics STM32U585 — Arm® Cortex®-M33 up to 160 MHz, 2 MB flash, 786 kB SRAM, FPU
MCU OS	Zephyr (runs Arduino sketches)
Wireless	WCBN3536A: Wi-Fi® 5 dual-band (2.4/5 GHz), Bluetooth® 5.1, onboard antenna
Expansion	Qwiic connector (Modulino), UNO headers (shields), bottom high-speed connectors (carriers: MIPI-CSI camera, MIPI-DSI display, analog audio)
RAM variants	2 GB / 4 GB (4 GB recommended for standalone SBC use)
Onboard LEDs	4 RGB user LEDs — #1 & #2 controlled by MPU, #3 & #4 by MCU
Power	USB-C (use a USB-C dongle with Power Delivery for standalone/desktop use — without PD the board will not boot)

Pinout / datasheet / schematics:

- Full pinout (PDF):
<https://docs.arduino.cc/resources/pinouts/ABX00162-full-pinout.pdf>
 - Datasheet:
<https://docs.arduino.cc/resources/datasheets/ABX00162-ABX00173-datasheet.pdf>
 - Schematics:
<https://docs.arduino.cc/resources/schematics/ABX00162-schematics.pdf>
-

Development Workflows

There are three main entry points. App Lab is recommended because it orchestrates both processors together.

Workflow A: Arduino App Lab (recommended)

App Lab is the integrated environment that builds the sketch, sets up the Python environment, installs the Bricks defined in the App, and launches both sides together.

1. Connect and configure

- Connect the UNO Q to your computer via USB-C (or use it standalone with an HDMI display, keyboard, and mouse through a PD dongle).
- On first use, follow the prompts to update the firmware, set a username/password, and configure Wi-Fi. This also enables an SSH server on the board.

2. Navigate the interface

The left sidebar provides: My Apps (your projects), Examples (built-in, read-only), Bricks (modular blocks), Learn (offline docs), and Settings (Wi-Fi, Linux password, keyboard layout).

3. Run a built-in example

- Open Examples → Blink LED and press Run. App Lab compiles the C++ sketch, builds the Python environment, and launches both. The Console panel opens at the bottom; when the Run button turns into Stop and a green notice appears, the App is live. LED3_R (an MCU-controlled LED) begins blinking.

4. Copy and edit

- Built-in examples are read-only. Choose Copy and edit app, name it, and Create new to get an editable copy. Use the File Manager in the sidebar to open `python/main.py` and `sketch/sketch.ino`.

5. Log and debug

- Standard Python `print()` statements are captured automatically and shown in the Python tab of the Console — the simplest way to trace behavior.

Minimal cross-processor example (Bridge)

MCU side ([sketch/sketch.ino](#)) — register a function the Linux side can call:

```
#include <Arduino.h>
#include "Bridge.h" // Arduino Bridge (RPC) library

const int LED_PIN = LED_BUILTIN;

// This function is exposed over RPC and called from Python (MPU)
void set_led_state(bool on) {
    digitalWrite(LED_PIN, on ? HIGH : LOW);
}

void setup() {
    pinMode(LED_PIN, OUTPUT);
    Bridge.begin();
    Bridge.provide("set_led_state", set_led_state); // service discovery
}

void loop() {
    Bridge.update();
}
```

MPU side ([python/main.py](#)) — call the MCU function over the Bridge:

```
import time
from arduino.app_bridge import Bridge # App Lab helper

bridge = Bridge()

state = False
while True:
    state = not state
    bridge.call("set_led_state", state) # remote call to the MCU
    print(f"LED set to {state}")      # shows in the Python console tab
    time.sleep(1)
```

API names above illustrate the pattern (register on one side, call from the other). Use the exact class/method names from the built-in Bridge examples in App Lab and the User Manual, as helper APIs may differ by App Lab version.

Workflow B: Arduino IDE / CLI (MCU only)

If you only need the microcontroller (classic Arduino experience):

- Use Arduino IDE 2.0+ to program the STM32U585 MCU directly in C/C++, or integrate Arduino CLI into editors like VS Code.
- This path does not orchestrate the Python/MPU side or Bricks — it is pure MCU development.

Workflow C: Standalone Linux (SBC mode) & CLI

Use the UNO Q like a Linux single-board computer:

- Attach an HDMI display, keyboard, and mouse via a PD-capable USB-C dongle, or connect over SSH once Wi-Fi is configured. VS Code + the Remote-SSH extension works well for browsing files on the board.

Drive the MPU-controlled LEDs directly from the Linux side, e.g. from the shell:

```
echo 1 | sudo tee /sys/class/leds/red:user/brightness # ON
echo 0 | sudo tee /sys/class/leds/red:user/brightness # OFF
```

-

Run Apps from the command line with the App Lab CLI:

```
arduino-app-cli app start ~/ArduinoApps/uno_q_blink # build + run both sides
arduino-app-cli app logs ~/ArduinoApps/uno_q_blink # view Python logs
arduino-app-cli app stop ~/ArduinoApps/uno_q_blink # stop (runs in background)
```

- The first run may take a while as Python/Arduino libraries install.

Additional References

- Hardware page: <https://docs.arduino.cc/hardware/uno-q/>
- UNO Q User Manual (Bridge/RPC, LEDs, SPI/I²C, standalone mode): <https://docs.arduino.cc/tutorials/uno-q/user-manual/>

- App Lab getting started:
<https://docs.arduino.cc/software/app-lab/tutorials/getting-started/>
 - Product overview & App Lab download: <https://www.arduino.cc/en/uno-q>
 - Qualcomm QRB2210 overview:
<https://www.qualcomm.com/products/internet-of-things/robotics-processors/qrb2210>
 - STM32U585 datasheet: <https://www.st.com/resource/en/datasheet/stm32u585ai.pdf>
 - Project Hub (examples): <https://projecthub.arduino.cc/?value=UNO%20Q>
 - Forum: <https://forum.arduino.cc/c/hardware/uno-familyd/uno-q/222>
-

Tip for teams: pick your track, start from the closest built-in Example, "Copy and edit," then pull in Bricks (Camera, Object Detection, Web Server) to add features fast. Keep heavy/AI/networking work in Python on the MPU and timing-critical hardware control in the sketch on the MCU, and let the Bridge move data between them.

AI Hub

Snapdragon Multiverse Hackathon: Qualcomm AI Hub Developer Guide

This guide covers the Qualcomm AI Hub workflow — how to find a model, get it compiled and quantized for a Snapdragon target, download the on-device bundle, and profile it — as one leg of your multi-device build for this hackathon (alongside the AI PC, mobile device, Arduino UNO Q, and Qualcomm Cloud AI 100).

“Source note: This guide uses only the public `qai-hub` / `qai-hub-models` Python packages and `onnxruntime` — no special client software is required. Every command here works from a plain `pip` install on your own laptop. For the canonical model catalog and account setup, always cross-check app.aihub.qualcomm.com.”

Table of Contents

- [Pre-work](#)
- [Core Architectural Concepts](#)
 - [What AI Hub Actually Does](#)
 - [The Build Host vs Run Host Split](#)
 - [Two Credentials You'll Need](#)
- [Track 1: Classical Models \(Vision/Audio\) via AI Hub](#)
- [Track 2: LLM / GenAI Bundles via AI Hub Genie](#)
- [Optimizing a Model for a Specific Device](#)
- [Profiling on Real Hardware](#)
- [Common Failure Modes](#)

- [Additional References](#)
-

Pre-work

1. Create an AI Hub account and get your API token at

app.aihub.qualcomm.com → Account → API Token.

Treat this token as a secret — never commit it, never paste it into a shared channel.

2. Install the AI Hub CLI/SDK:

```
pip install qai-hub qai-hub-models
qai-hub configure --api_token <your-token>
```

- 3.

4. Know which chipset you're targeting. AI Hub compiles per exact device —

```
qai-hub list-devices | grep -i "<your device>"
```

- 5.

For this hackathon that's most likely your AI PC (Snapdragon X Elite) or your phone (Snapdragon 8 Elite).

6. Decide classical model vs LLM early — the two paths use different AI Hub export flows and have very different resource requirements (see Tracks below).

7. If you're on Windows-ARM (Snapdragon X Elite) already — read [The Build Host vs Run Host Split](#) before you try to build anything. This is the single most common setup mistake.
-

Core Architectural Concepts

What AI Hub Actually Does

Qualcomm AI Hub is a cloud compilation service. You give it a model (ONNX / PyTorch / TFLite) and a target device string, and it:

1. Converts and quantizes the model for that device's exact NPU/DSP version
2. Runs it on real cloud-hosted Snapdragon hardware and reports back real latency
3. Returns a downloadable, ready-to-run artifact (QNN context binary, or for LLMs a full Genie bundle with tokenizer + config)

This is different from converting locally with the QAIRT SDK on your own machine (`qairt-converter` / `snpe-onnx-to-dlc`, no cloud round-trip) — AI Hub does the compile in the cloud, against real target hardware, which is why it needs your API token and a network connection.

The Build Host vs Run Host Split

This is the fact that trips up the most teams, so read it twice:

“The machine that exports an LLM bundle and the machine that runs it are usually different machines — even though they’re often the same Snapdragon laptop.”

Why: the AI Hub LLM exporter (qai-hub-models) depends on torch, and torch has no Windows-ARM64 wheel. So you cannot run the exporter directly on a Snapdragon X Elite's native Windows-ARM64 Python — even though the X Elite is exactly where the resulting bundle runs.

Host type	Can it BUILD (export) a bundle?	Can it RUN a bundle?
Windows-ARM64 (X Elite, native Python)	✗ no torch wheel	✓ yes — this is the target
WSL2 on the same X Elite (arm64 Linux)	✓ torch has arm64 Linux wheels	— (copy bundle out to Windows side)
x86_64 Windows / macOS / Linux dev box	✓	—

Practical flow: export the bundle in WSL (or any x86/macOS/Linux box), then copy the output folder onto the X Elite's Windows filesystem and run it there with the native ARM64 Python + onnxruntime-genai.

To set up WSL on your X Elite laptop (one-time, admin PowerShell):

```
wsl --install
```

Then inside WSL:

```
python3 -m venv ~/aihub-build && source ~/aihub-build/bin/activate  
pip install -U qai-hub qai-hub-models
```

This split only applies to LLM/Genie bundle exports. Classical vision/audio models compiled via `qai_hub_models.models.<slug>.export --target-runtime qnn` don't need `torch` locally in the same heavy way — see [Track 1](#).

Two Credentials You'll Need

Credential	Needed for	Where to get it
AI Hub API token	Every AI Hub compile/export — classical or LLM	app.aihub.qualcomm.com → Account → API Token
Hugging Face token	Only for gated LLM weights (Llama, Qwen)	huggingface.co — accept the model's license on its page first, then generate a token

Open models need no HF token at all. If your team wants to move fast on the LLM track without dealing with license acceptance, start with an open model (e.g. Falcon) to prove the pipeline end-to-end before touching gated weights.

Configure the AI Hub token once (on whichever machine will run exports):

```
export QAI_HUB_API_TOKEN=<your-token>
qai-hub configure --api_token $QAI_HUB_API_TOKEN
```

For gated LLM weights, also set:

```
export HF_TOKEN=<your-huggingface-token>
```

Track 1: Classical Models (Vision/Audio) via AI Hub

Best for: image classification, object detection, segmentation, audio classification — models that don't need the LLM decode-loop machinery.

1. Find a model

Browse the catalog: app.aihub.qualcomm.com →

Models. Filter by task (classification, detection, segmentation) and check that a precompiled artifact exists for your target device.

2. Export/compile for your device

```
python -m qai_hub_models.models.<slug>.export \
  --target-runtime qnn \
  --device "<exact device label from qai-hub list-devices>"
```

This runs the compile in the cloud against real hardware and downloads the result — you'll get real, hardware-measured latency numbers back, not an estimate. Note the output path it prints; that's your compiled artifact.

3. Bring it into your app

The downloaded artifact is a QNN context binary. Load it directly via `onnxruntime` with the QNN execution provider:

```
import os, onnxruntime as ort

# The QNN EP ships alongside onnxruntime-qnn (pip install onnxruntime-qnn)
sess = ort.InferenceSession(
    "model.onnx",
    providers=["QNNExecutionProvider"],
    provider_options=[{"backend_path": "QnnHtp.dll"}], # libQnnHtp.so on Linux/Android
)
outputs = sess.run(None, {"input": your_input_array})
```

If you'd rather convert a model you already have (ONNX/PyTorch) locally instead of a cloud round-trip, use the QAIRT SDK directly:

```
qairt-converter --input_network model.onnx --output_path model_int8.dlc \
--input_list calibration_images.txt --quantization_overrides int8
```

4. Profile it

See [Profiling on Real Hardware](#) below.

Track 2: LLM / GenAI Bundles via AI Hub Genie

Best for: on-device chat, summarization, function-calling, or any generative text use case running locally on the NPU (no cloud inference call at runtime).

1. Set up a build host

Per [The Build Host vs Run Host Split](#), do this

in WSL (if you're on the X Elite) or any x86/macOS/Linux box:

```
python3 -m venv ~/aihub-build && source ~/aihub-build/bin/activate
pip install -U qai-hub qai-hub-models
export QAI_HUB_API_TOKEN=<token>
qai-hub configure --api_token $QAI_HUB_API_TOKEN
```

2. Find your exact device string and export

```
qai-hub list-devices | grep -i "<your device>"

python -m qai_hub_models.models.<slug>.export \
  --target-runtime genie \
  --device "<exact device label>"
```

Use the model's package slug (underscores), not its friendly display name — e.g. `qai_hub_models.models.falcon_v3_7b_instruct.export` for "Falcon v3 7B Instruct". Note the output directory — it contains `genai_config.json`, the compiled graph/context binary, and the tokenizer.

Gated weights (Llama, Qwen): accept the license on the model's Hugging Face page and export HF_TOKEN before running the export. Open models (e.g. Falcon) need no HF token — the best first model to validate your pipeline end-to-end.

This export is large (multi-GB download + compile time) — budget time for it, ideally the night before rather than during your build window.

3. Copy the bundle to the run host

If you built in WSL, the Windows filesystem is under /mnt/c:

```
DST=/mnt/c/<your-app>/models/<friendly-model-id>
mkdir -p "$DST"
cp -r <export-output-dir>/ * "$DST"/
```

For any other build/run host split (e.g. built on a teammate's x86 laptop, running on your X Elite), just copy the folder over any way you like — USB drive, shared network folder, scp if both machines are reachable over SSH. The destination folder name should be your model's friendly id — whatever your runtime code looks up by.

4. Load and run it on the NPU

Using onnxruntime-genai (install with pip install onnxruntime-genai on the run host — native ARM64 Python on an X Elite, not an emulated x86 shell):

```

import onnxruntime_genai as og

model = og.Model("/path/to/bundle_dir") # the folder with genai_config.json
tokenizer = og.Tokenizer(model)

prompt = "What is the Snapdragon X Elite?"
tokens = tokenizer.encode(prompt)

params = og.GeneratorParams(model)
params.set_inputs(tokens)
params.set_search_options(max_length=200)

generator = og.Generator(model, params)
while not generator.is_done():
    generator.compute_logits()
    generator.generate_next_token()
    print(tokenizer.decode([generator.get_next_tokens()[0]]), end="", flush=True)

```

5. Verify it actually runs on the NPU

Don't assume from a clean load — check the real execution provider directly:

```

import os, onnxruntime_qnn as q, onnxruntime as o
os.add_dll_directory(os.path.dirname(q.__file__))
o.register_execution_provider_library("QNNExecutionProvider", q.get_library_path())
npu_devices = [d for d in o.get_ep_devices()
                if d.ep_name == "QNNExecutionProvider" and str(d.device.type).endswith("NPU")]
print("NPU device found:", bool(npu_devices))

```

If it falls back to CPU silently, your demo will look "done" but won't show the actual NPU speedup judges are scoring on Technical Implementation (40 points — explicitly includes "resource utilization, optimization, latency and performance, and energy efficiency"). Always verify the execution provider

directly rather than trusting that "it loaded" means "it's on the NPU." Note that in `onnxruntime-qnn 2.x`, the QNN EP does not show up in `onnxruntime.get_available_providers()` even when it's working correctly — you must check via `get_ep_devices()` as shown above.

Optimizing a Model for a Specific Device

Getting a model to *compile* for a device is the easy part — `--device "<label>"` handles that. Getting it to actually run well on that device's specific NPU/DSP generation is a separate set of choices. This is also exactly what the Technical Implementation judging criterion (40 points — "resource utilization, optimization, latency and performance, and energy efficiency") is scoring, so it's worth doing properly rather than just accepting AI Hub's defaults.

1. Match quantization to the device's NPU generation

Different Hexagon NPU versions have different native precision support. Compiling FP32 to a device whose NPU only accelerates INT8 means large parts of your graph silently fall back to CPU — slow, and higher power draw.

Device (this hackathon)	NPU	Native precision	Recommended quantization
Snapdragon X Elite (AI PC)	Hexagon v73, 45 TOPS	INT8 (INT4 for weights on LLMs)	INT8 for vision/audio; INT4 weight-only + FP16 activations for LLMs

Snapdragon 8 Elite (mobile)	Hexagon v73–v81 depending on exact chip	INT8	INT8
Arduino UNO Q (QCS2210)	Hexagon DSP v66, 1.0 TOPS	INT8 only	INT8 — mandatory; FP32 does not run on this DSP generation at all and silently falls back to CPU

Rule of thumb: always quantize to at least INT8 before compiling for any of this hackathon's NPU targets. INT8 typically gives $\geq 4\times$ size reduction with $< 1\%$ accuracy loss on classical vision models. Only go to INT4 for LLM weights (not classical vision models), and only if you can tolerate a larger accuracy drop.

2. Provide real calibration data — not random noise

Static quantization needs a calibration pass to compute per-tensor scale/zero-point values. If you calibrate with random data, your scales will be wrong and accuracy silently degrades even though the pipeline "works":

```
python -m qai_hub_models.models.<slug>.export \
  --target-runtime qnn \
  --device "<device label>" \
  --calibration-data ./calibration_images/ # real, representative inputs
```

Or with `onnxruntime.quantization.quantize_static` locally, pass a `CalibrationDataReader` backed by real sample inputs rather than random arrays.

Use ~10–100 real samples that match your actual inference-time input distribution (same resolution, same preprocessing, same domain — e.g. actual crowd photos if your model classifies crowd density, not generic ImageNet samples).

3. Check op coverage — don't assume 100% lands on the NPU

Most AI Hub export commands print a per-op breakdown of what compiled onto the target NPU vs what will fall back to CPU. Read this output — don't skip past it:

- $\geq 80\%$ coverage $\rightarrow$ NPU is still the right call; the few CPU-fallback ops won't dominate latency.
- $< 80\%$ coverage $\rightarrow$ the CPU $\leftrightarrow$ NPU round-trips for fallback ops may cost more than they save. Consider running on GPU instead (no CPU $\leftrightarrow$ NPU boundary crossing cost), or restructure the model to avoid the unsupported op (e.g. swap a custom activation for a standard one).

You can also inspect this directly with `onnxruntime`'s session after loading — `sess.get_providers()` shows which providers are actually attached, and verbose logging (`ort.set_default_logger_severity(0)`) will print the partition map showing which nodes were assigned to which execution provider.

4. Pick a power mode that matches your use case, not just "fastest"

Use case	Recommended profile	Why

Real-time camera/video pipeline	burst / performance	Pins NPU clock, avoids frame-to-frame latency variance from thermal throttling
Single interactive inference (e.g. chat turn, one photo)	balanced	Good latency without sustained high power draw
Battery-powered / sustained background sensing	power_saver / efficiency	Lowest power; latency variance is acceptable since nothing is real-time

(Exact flag names vary by tool — `snpe-net-run --perf_profile <mode>`, `qnn-net-run --perf_profile <mode>`; check `--help` on whichever CLI you're using.)

Running everything in `performance/burst` mode for a demo will look fast, but a judge asking about energy efficiency (explicitly scored) will catch a mismatch between your use case and your power profile choice — pick deliberately and be ready to explain why.

5. For LLMs specifically: profile per-token, not per-inference

A single "latency" number is meaningless for generative text. Report:

- Time-to-first-token (prefill latency) — matters for perceived responsiveness
- Tokens/sec during decode (steady-state generation speed) — matters for how fast the full response streams in

Both numbers should be captured from the actual device, not the AI Hub cloud compile-time estimate — see [Profiling on Real Hardware](#).

Profiling on Real Hardware

Once your model is compiled and deployed, get real numbers for your demo and submission writeup — don't rely on AI Hub's cloud compile-time estimate alone; capture numbers from the actual device you'll demo on.

Simple timing in Python (works anywhere `onnxruntime` runs):

```
import time
import numpy as np

# warmup
for _ in range(3):
    sess.run(None, {input_name: sample_input})

times = []
for _ in range(50):
    t0 = time.perf_counter()
    sess.run(None, {input_name: sample_input})
    times.append((time.perf_counter() - t0) * 1000)

times = np.array(times)
print(f"mean={times.mean():.1f}ms p50={np.percentile(times,50):.1f}ms "
      f"p95={np.percentile(times,95):.1f}ms p99={np.percentile(times,99):.1f}ms")
```

On the Arduino UNO Q board specifically, use the SNPE CLI's built-in benchmarking (see the [UNO Q guide](#) for the full on-device workflow):

```
snpe-net-run --container model_int8.dlc --input_list inputs.txt \
--use_dsp --duration 30 --debug
```


`--debug` prints per-layer timing; parse `Total Inference` for end-to-end latency.

For a visual profile (Chrome trace of kernel-level execution), QNN's `qnn-profile-viewer` tool (bundled with the QAIRT SDK) converts a profiling log into a trace you can drag into `chrome://tracing`.

Capture at minimum: mean/p50/p95/p99 latency, throughput (inferences/sec or tokens/sec), and — if you can measure it (many dev boards expose this via `/sys/class/power_supply/` or a USB power meter) — power draw. This is exactly what the Technical Implementation judging criterion is scoring, so having real numbers (not estimates) for your presentation matters.

Common Failure Modes

Symptom	Cause → Fix
No matching distribution found for <code>onnxruntime-genai-qnn</code>	That package name doesn't exist. The correct package is <code>onnxruntime-genai</code> — its <code>win-arm64</code> wheel already carries Hexagon support.
<code>pip</code> can't find a <code>win_arm64</code> wheel for <code>torch</code>	You're trying to build an LLM bundle directly on the X Elite. Build in WSL/x86/macOS/Linux

	instead — see Build Host vs Run Host.
quantized checkpoint ... must have an ONNX model	Wrong export flag. LLMs need <code>--target-runtime genie</code>, not the vision <code>--target-runtime qnn</code>.
403 / access denied during export	Gated weights. Accept the license on the model's Hugging Face page and set <code>HF_TOKEN</code>, or use an open model like Falcon.
Execution provider reports CPU after a "successful" load	The QNN EP library wasn't registered/attached correctly. Re-check the verification snippet in Track 2 step 5 — don't trust "it loaded" as "it's on the NPU."
<code>onnxruntime.get_available_providers()</code> doesn't list QNN even though the NPU works	Expected in <code>onnxruntime-qnn 2.x</code> — it's a plugin EP and won't show there. Check via <code>get_ep_devices()</code> instead (see Track 2 step 5).
Export/compile takes forever or times out	AI Hub queues jobs against real hardware — this is a shared resource during the hackathon. Kick off large exports early and in parallel with other work.

Additional References

- Official AI Hub portal (account, token, model catalog, device list):
app.aihub.qualcomm.com
- `qai-hub` / `qai-hub-models` PyPI packages and their own `--help` output are the most current reference for exact CLI flags — they change between SDK versions.
- Qualcomm SNPE / QNN SDK developer docs (for local, non-cloud conversion)
- Companion guide for the Arduino UNO Q leg of this hackathon:
`docs/UNOQ_HACKATHON_DEVELOPER_GUIDE.md`